

Trabalho Laboratorial 1

Relatório da Unidade Curricular de Visão Computacional
Mestrado em Engenharia Eletrotécnica
2024/2025

Autores:

	
Nome: João Paulino Nº de Estudante: 2230075	Nome: Judá Imbó Nº de Estudante: 2232626

Índice

Índice	III
Índice de Figuras	IV
Advertência	5
1. Introdução	6
2. Extração do tempo analógico por escala de cinzas	7
3. Extração do tempo por escala de cores	14
4. Execução do algoritmo em vídeo	18
5. Conclusão	20

Índice de Figuras

Figura 1 – Imagem original em escala de cinzas.....	7
Figura 2 – Código para a obtenção da máscara para a região de interesse	7
Figura 3 – Máscara da região de interesse (à esquerda) e imagem original vista dentro da região de interesse (à direita)	8
Figura 4 – Resultado da utilização do <i>threshold</i>	8
Figura 5 – Imagem após a utilização da função “GaussianBlur”	9
Figura 6 – imagem após a utilização da função “Canny”	9
Figura 7 – Linhas detetadas na região de interesse a branco	10
Figura 8 – Função “agrupar_linhas”.....	10
Figura 9 – Função “get_hands”	11
Figura 10 – Função “get_angle”	12
Figura 11 - Resultado final da deteção dos ponteiros pela escala de cinzas	13
Figura 12 - Imagem em escala S	14
Figura 13 – Imagem em escala V	14
Figura 14 – Imagem em escala S dentro da região de interesse	15
Figura 15 – Imagem em escala V dentro da região de interesse.....	15
Figura 16 – Imagens da escalas S e V após a realização do <i>Threshold</i>	15
Figura 17 – Imagens das escalas S e V após a utilização da função “Canny”	16
Figura 18 – Linhas detetadas nas imagens da escala S (à direita) e da escala V (à esquerda).....	16
Figura 19 – Aplicação da <i>flag</i> “ponteiro_segundos”	17
Figura 20 – função “get_hands_V”	18
Figura 21 – Alternativa utilizada na função “agrupar_linhas”	18
Figura 22 – Chamada da função “Canny” com os limiares máximos alterados	19
Figura 23 – Ficheiro .csv com as frames e o tempo obtido	19

Advertência

Por favor, antes de executar o programa submetido leia os ficheiros “README.md”, para uma fácil execução do programa.

1. Introdução

Este relatório, referente ao trabalho laboratorial 1 da Unidade curricular de Visão Computacional, tem como objetivo a exploração e compreensão das matérias relacionadas com o processamento de imagem e a extração de informação a partir de diferentes escalas de cor. O foco principal está na conversão de tempo de um relógio analógico para digital. Para o desenvolvimento deste trabalho, foi utilizado o *software* Visual Studio Code em conjunto com a linguagem de programação Python e a biblioteca *open-source* OpenCV, utilizada para trabalhos de visão computacional e processamento de imagem.

O relatório se encontra organizado em vários capítulos com as metodologias e os algoritmos desenvolvidos. No capítulo 2 descreve a extração do tempo em escala de cinzas, enquanto o capítulo 3 aborda a extração, mas em escala de cores HSV. O capítulo 4 explica a execução do algoritmo em vídeo e as modificações que foram necessárias para o normal funcionamento do programa e, por fim, uma breve conclusão é dada no capítulo 5.

2. Extração do tempo analógico por escala de cinzas

O primeiro passo para a obtenção do tempo através da escala de cinzas, foi a conversão da imagem originalmente lida em BGR para a escala de cinzas através da função “cvtColor”, com o parâmetro “COLOR_BGR2GRAY”.



FIGURA 1 – IMAGEM ORIGINAL EM ESCALA DE CINZAS

De seguida, foi criada uma região de interesse com objetivo de facilitar a detecção dos ponteiros dentro um espaço específico. Para tal, a função “HoughCircles” foi usada com parâmetros para limitar o raio mínimo e máximo dos círculos em pixels. Esta função por sua vez, cria um *array* de círculos detetados com as suas coordenadas do centro do círculo e o seu raio. Este *array* é usado na função “detect_region_of_interest”, no qual esta utiliza um ciclo “for” para obter o maior círculo através do raio e cria uma máscara com a região de interesse pintada a branco, tal como mostra a figura 2. Os resultados se encontram na figura 3.

```
#Afunção cv2.circle apenas recebe os parâmetros do centro e do raio com tipo 'int', como a função cv2.circle
#precisa de valores da posição do centro do círculo e do raio a inteiro
pos_cir_x = int(maior_circulo[0])
pos_cir_y = int(maior_circulo[1])
raio_cir = int(maior_circulo[2])

# Dsenha o maior círculo e define a região de interesse à volta da área de relógio (denominado de mask)
mask = np.zeros_like(image)

# Draw a filled white circle on the mask
cv2.circle(mask, (pos_cir_x, pos_cir_y), raio_cir, (255, 255, 255), thickness=-1)

# Apply the mask to the original grayscale image using bitwise AND
masked_image = cv2.bitwise_and(image, image, mask=mask)
```

FIGURA 2 – CÓDIGO PARA A OBTENÇÃO DA MÁSCARA PARA A REGIÃO DE INTERESSE

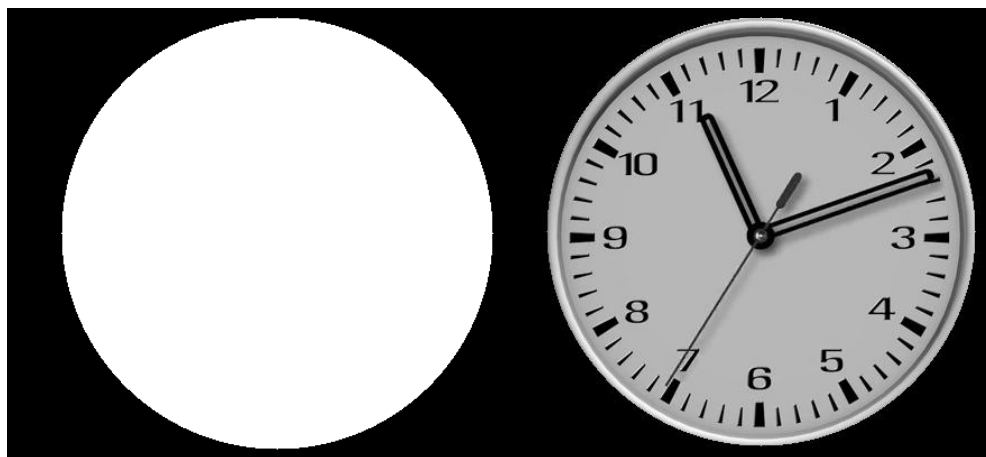


FIGURA 3 – MÁSCARA DA REGIÃO DE INTERESSE (À ESQUERDA) E IMAGEM ORIGINAL VISTA DENTRO DA REGIÃO DE INTERESSE (À DIREITA)

Com uma região de interesse, foi possível a detecção dos ponteiros. A função “detect_ponteiros”, mostra todo o processo para detetar cada ponteiro. O método de obtenção foi adaptado do trabalho que se encontra no seguinte link: [\[aqui\]](#). O passo inicial está relacionado com o processamento da imagem:

1. Foi realizado um *threshold* da imagem para a converter em uma imagem binária, usando o método de linearização Otsu com inversão binária (“cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU”);

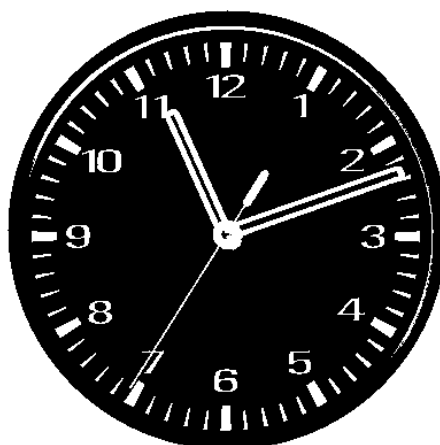


FIGURA 4 – RESULTADO DA UTILIZAÇÃO DO *THRESHOLD*

2. Reduziu-se o ruído na imagem com um filtro gaussiano de tamanho 5x5 através da função “GaussianBlur”;



FIGURA 5 – IMAGEM APÓS A UTILIZAÇÃO DA FUNÇÃO “GAUSSIANBLUR”

3. Fez-se a detecção de bordas na imagem usando a função “Canny”;

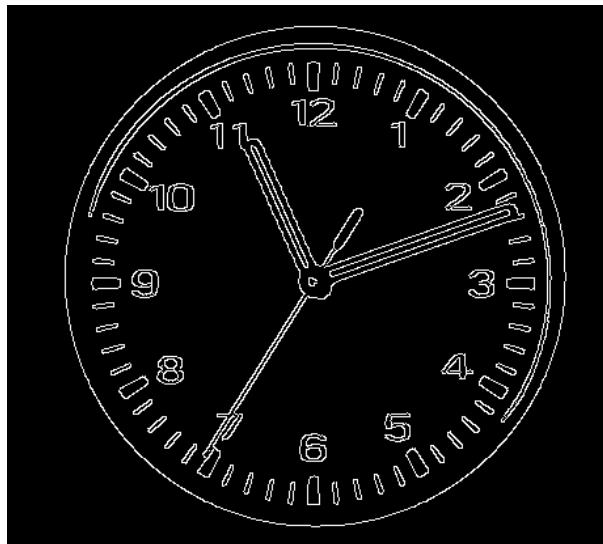


FIGURA 6 – IMAGEM APÓS A UTILIZAÇÃO DA FUNÇÃO “CANNY”

4. Por fim, foi criado um array de linhas detetadas pela função “HoughLinesP”, que usa a transformada de Hough Probabilística com parâmetros de comprimento mínimo e de distância máxima entre segmentos de linhas.

Como resultado, foi desenhada as linhas detetadas dentro da região de interesse a branco na figura seguinte.

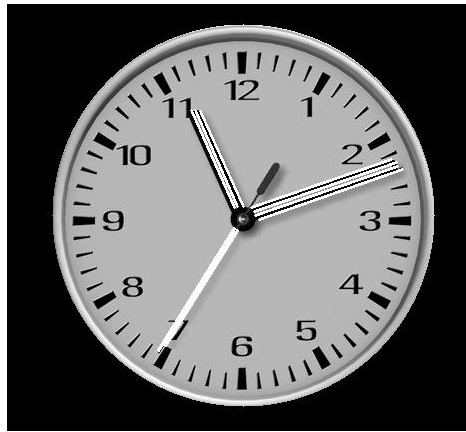


FIGURA 7 – LINHAS DETETADAS NA REGIÃO DE INTERESSE A BRANCO

O passo seguinte passa pela função “agrupar_linhas”, no qual agrupa as linhas detetadas anteriormente, através da orientação (“angle_threshold”) e da proximidade entre linhas (“proximity_threshold”). A orientação representa a máxima diferença em graus entre duas linhas para serem consideradas paralelas e para serem agrupadas juntas. A proximidade entre linhas representa a distância máxima entre os centroides de duas linhas para serem agrupadas. Caso estes parâmetros tiverem valores elevados, vão apanhar um maior número de linhas com grandes variações de ângulo e de distância entre linhas. Caso contrário, os grupos vão estar mais restritos, em termos de distância (as linhas irão estar mais próximas) e em termos de ângulos (as linhas estarão mais perto de serem realmente paralelas). A figura 8 mostra o conteúdo desta função.

```
grupos= []

for line in lines:
    x1, y1, x2, y2 = line[0]

    distancia1 = np.sqrt((x1 - pos_cir_x)**2 + (y1 - pos_cir_y)**2)
    distancia2 = np.sqrt((x2 - pos_cir_x)**2 + (y2 - pos_cir_y)**2)

    if((max(distancia1,distancia2) < raio) and (min(distancia1,distancia2) < raio*50/100)):
        # Calculate line angle in degrees and round to nearest multiple of angle_threshold
        angle = math.atan2(y2 - y1, x2 - x1)
        angle_deg = round(math.degrees(angle) / angle_threshold) * angle_threshold

        # Calculate line centroid (midpoint) for distance comparisons
        centroid = ((x1 + x2) / 2, (y1 + y2) / 2)

        # Check if line belongs to an existing group based on angle and proximity
        grouped = False
        for group in grupos:
            if abs(angle_deg - group['angle']) < angle_threshold:
                # Check proximity to group's centroid
                group_centroid = group['centroid']
                distance = np.sqrt((centroid[0] - group_centroid[0])**2 + (centroid[1] - group_centroid[1])**2)
                if distance < proximity_threshold:
                    # Add line to group and update group centroid
                    group['lines'].append(line)
                    group['centroid'] = ((group_centroid[0] + centroid[0]) / 2, (group_centroid[1] + centroid[1]) / 2)
                    grouped = True
                    break

        if not grouped:
            grupos.append({'lines': [line], 'angle': angle_deg, 'centroid': centroid})

    else:
        centroid = ((x1 + x2) / 2, (y1 + y2) / 2)
        # Handle special case where condition is not met
        grupos.append({'lines': [line], 'angle': 0, 'centroid': centroid})

return grupos
```

FIGURA 8 – FUNÇÃO “AGRUPAR_LINHAS”

A função “agrupar_linhas” devolve um array de grupos de linhas que posteriormente é usada pela função “detetar_ponteiros”, no qual procura em cada grupo o ponto mais longe do centro do círculo (“max_line”), a maior espessura (“max_thickness”) e o maior comprimento (“max_length”) para definir uma linha. No final desta função, as linhas são organizadas por comprimento em ordem decrescente e devolve os três primeiros ponteiros.

De seguida, é utilizada a função para identificar os ponteiros denominada “get_hands”, demonstrada na figura 9. Esta função utiliza os ponteiros devolvidos da função “agrupar_linhas” e as organiza em grossura (para identificar o ponteiro dos segundos) e posteriormente em comprimento, organizando-os em ordem crescente para identificar em primeiro lugar, o ponteiro dos minutos e depois o ponteiro das horas.

```
# Arrange the clock hands by thickness
sorted_hands_by_thickness = sorted(hands, key=lambda hands: hands[1])

if len(sorted_hands_by_thickness)==1:
    if len(sorted_hands_by_thickness[0][0])==4 and not isinstance(sorted_hands_by_thickness[0][1], list) and not isinstance(sorted_hands_by_thickness[0][2], list):
        reorder_hands = [sorted_hands_by_thickness[0][0], sorted_hands_by_thickness[0][1], sorted_hands_by_thickness[0][2]]
        test1=True
        second_hand = reorder_hands
        hour_hand = reorder_hands
        minute_hand = reorder_hands
    else:
        test1=False
        # The second hand is the hand with the smallest thickness
        second_hand = sorted_hands_by_thickness[0]

        # Remove the second hand from the list containing 3 clock hands
        hands.remove(second_hand)

        # Arrange the remaining 2 clock hands by length
        sorted_hands_by_length = sorted(hands, key=lambda hands: hands[2])

        # The hour hand is the hand with the shortest length and the remaining hand is the minute hand
        hour_hand = sorted_hands_by_length[0]

        #minute_hand = sorted_hands_by_length[1]

        if len(sorted_hands_by_length)==1:
            test2=True
            minute_hand = sorted_hands_by_length[0]
            # The hour hand is the hand with the shortest length and the remaining hand is the minute hand
            hour_hand = sorted_hands_by_length[0]
        else:
            test2=False
            hour_hand = sorted_hands_by_length[0]
            minute_hand = sorted_hands_by_length[1]

return hour_hand, minute_hand, second_hand
```

FIGURA 9 – FUNÇÃO “GET_HANDS”

Com os ponteiros identificados, foi possível descobrir a que horas correspondia a imagem captada ao descobrir o ângulo de cada ponteiro, através da função “get_angle” e o tempo a que corresponde pela função “get_time”. A função “get_angle” utiliza o vetor direcional do ponteiro e cria um vetor direcional horizontal a partir do centro do relógio. Com estes dois vetores, calcula o produto escalar entre eles e o comprimento de cada um para descobrir o cosseno do ângulo a partir da seguinte fórmula:

$$\cos(\theta) = \frac{u \cdot v}{|u| * |v|}$$

Por fim, é convertido o ângulo de radianos para graus e consoante o resultado do produto vetorial, esta função devolvia o valor do ângulo numa gama de 0 a 360 graus.

```

# u is the direction vector of the clock hands
u = get_vector(hand)

# Create a horizontal direction vector from the center of the clock
v = [center_x - center_x, center_y - (center_y-100)]

# Call the function to calculate the dot product of two vectors
dot_uv = dot_product(u, v)

# Calculate the length of vector u and v
length_u = math.sqrt(u[0]**2 + u[1]**2)
length_v = math.sqrt(v[0]**2 + v[1]**2)

# Calculate the cosine of the angle between two vectors using the formula u.v / (|u| * |v|)
cos_theta = dot_uv / (length_u * length_v)

# Limit the value of cos to the range [-1, 1] to avoid errors when calculating arccos
cos_theta = max(min(cos_theta, 1.0), -1.0)

# Calculate the angle using the formula arccos(cos_theta)
theta = math.acos(cos_theta)

# Convert angle from radians to degrees
theta_degrees = math.degrees(theta)

# If the directional product is greater than 0, that means vector u is to the left of vector v
# Conversely, if the directional product is less than or equal to 0, that means vector u is to the right or in the same direction as vector v
cross_uv = cross_product(u, v)
if cross_uv > 0:
    # Returns the complementary angle of theta
    return 360 - theta_degrees
else:
    return theta_degrees

```

FIGURA 10 – FUNÇÃO “GET_ANGLE”

A função “get_time”, utiliza os ângulos obtidos das horas, dos minutos e dos segundos e calcula o tempo correspondido da seguinte forma:

$$\begin{aligned}
 \text{hora} &= \frac{\text{ângulo das horas}}{30} \\
 \text{minuto} &= \frac{\text{ângulo dos minutos}}{6} \\
 \text{segundo} &= \frac{\text{ângulo dos segundos}}{6}
 \end{aligned}$$

É utilizado o valor “30” no cálculo das horas, pois cada hora corresponde a 30 graus (360 graus / 12 horas). Os minutos e os segundos são calculados da mesma forma, uma vez que cada minuto ou segundo corresponde a 6 graus (360 graus / 60 minutos ou segundos). Esta função também tem alguns ajustes para cada ponteiro para evitar erros ,caso os ponteiros estejam perto dos 0 ou dos 360 graus. No final desta função, é devolvido uma string com as horas calculadas no formato ‘HH:MM:SS’.

Tendo as horas calculadas, é desenhada na imagem original 3 retângulos a identificar cada ponteiro através da função “draw_ponteiros_frame” e as horas a partir da função “draw_time”, tal como mostra a figura 11.

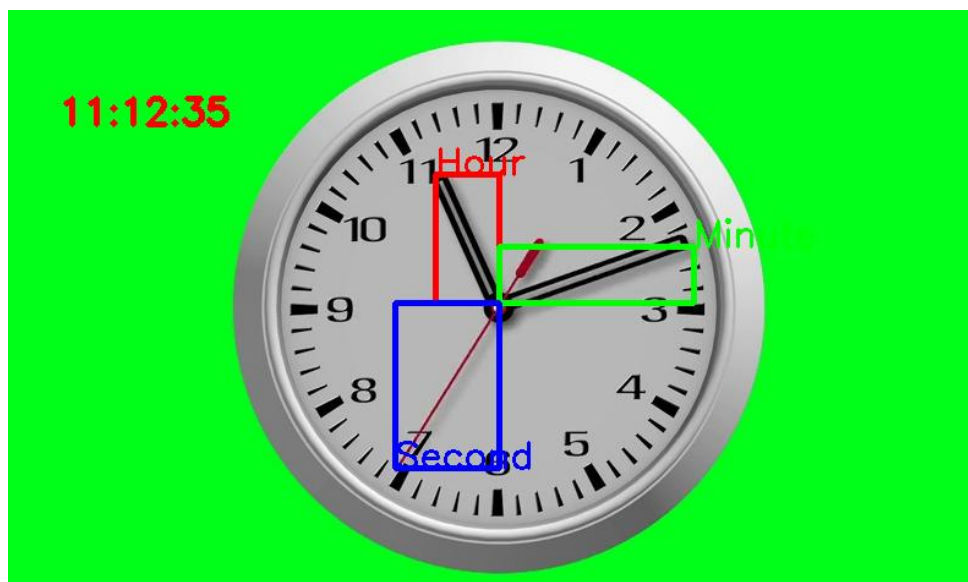


FIGURA 11 - RESULTADO FINAL DA DETEÇÃO DOS PONTEIROS PELA ESCALA DE CINZAS

3. Extração do tempo por escala de cores

Para extrair o tempo a partir da escala de cores HSV, foi necessário ter de separar cada parâmetro, tendo sido utilizado a imagem em escala S para a deteção do ponteiro dos segundos e a imagem em escala V para a deteção dos ponteiros das horas e dos minutos, tal como indica as figuras 12 e 13.

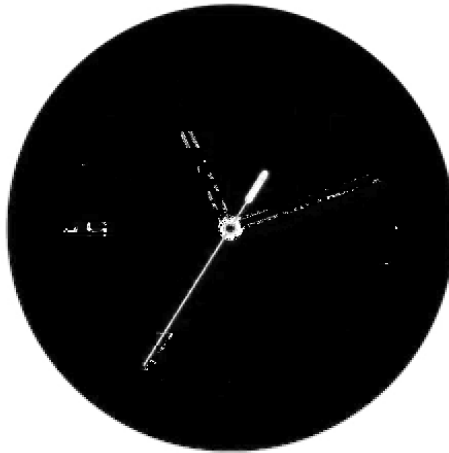


FIGURA 12 - IMAGEM EM ESCALA S



FIGURA 13 – IMAGEM EM ESCALA V

Foi utilizado o mesmo método para detetar a região de interesse da escala de cinzas, com a imagem em escala V devido às semelhanças entre imagens. Para a imagem em escala S, uma vez que são conhecidos os pontos do centro do círculo e o raio através da escala de cinzas, foi fácil a criação de uma região de interesse para esta imagem, tendo como resultado a figura seguinte.

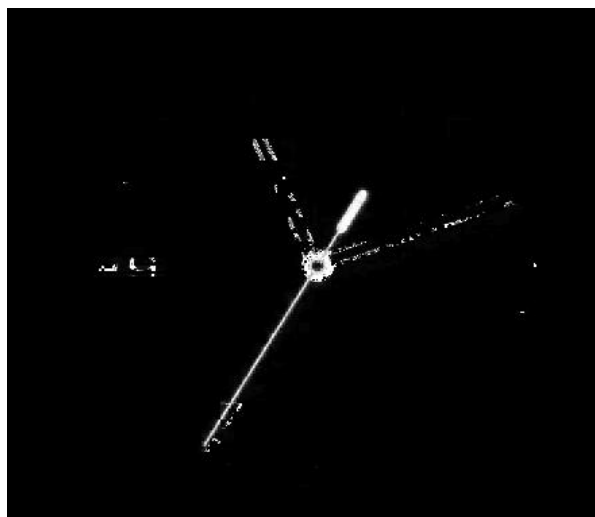


FIGURA 14 – IMAGEM EM ESCALA S DENTRO DA REGIÃO DE INTERESSE



FIGURA 15 – IMAGEM EM ESCALA V DENTRO DA REGIÃO DE INTERESSE

Também foram utilizadas as funções “GaussianBlur”, “threshold” e “Canny” para o processo de processamento das imagens, tendo os seguintes resultados demonstrados nas figuras 16 e 17.

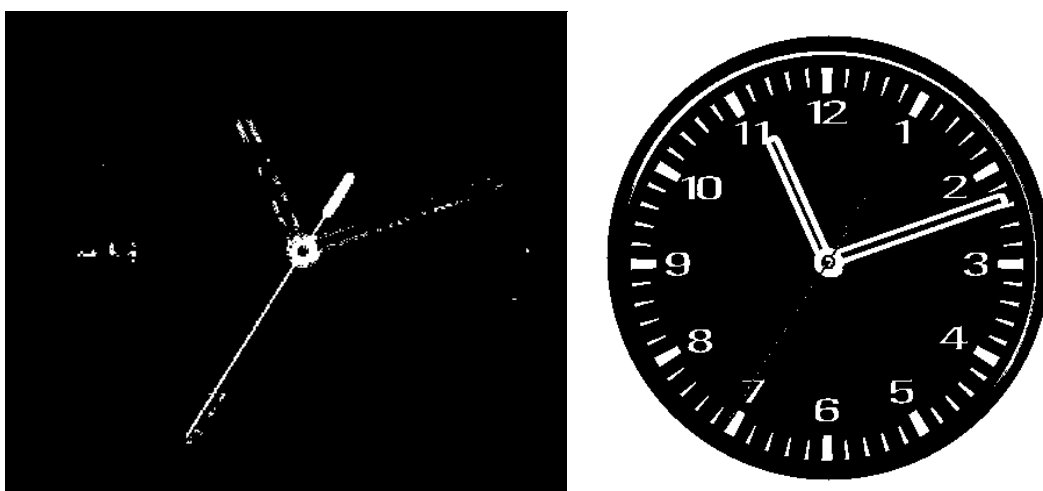


FIGURA 16 – IMAGENS DA ESCALAS S E V APÓS A REALIZAÇÃO DO *THRESHOLD*

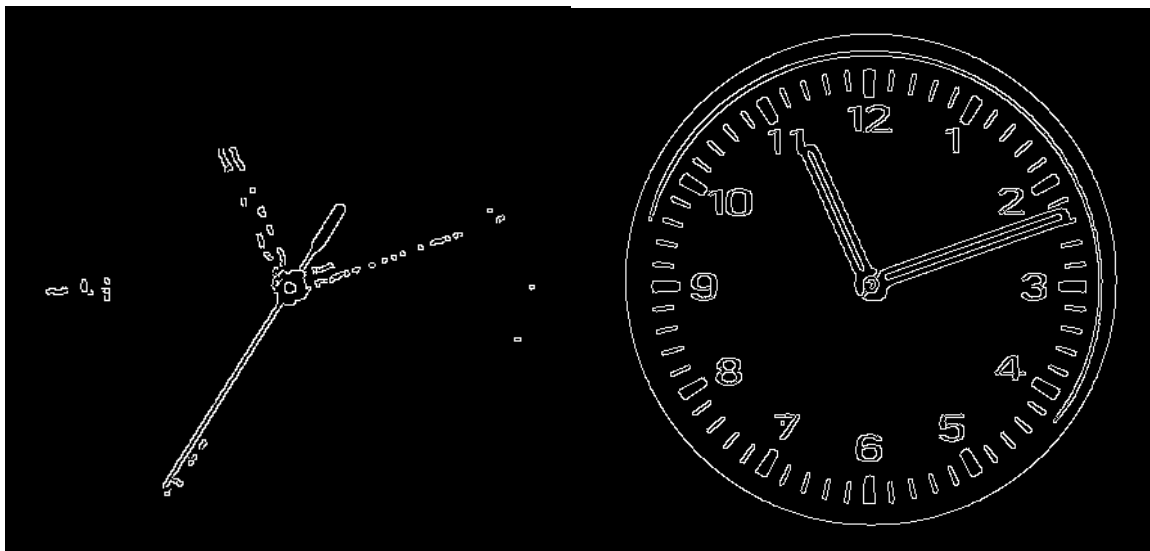


FIGURA 17 – IMAGENS DAS ESCALAS S E V APÓS A UTILIZAÇÃO DA FUNÇÃO “CANNY”

O método e as funções para a detecção das linhas e o agrupamento das mesmas nas escalas S e V continuam a ser as mesmas utilizadas para a escala de cinzas, só que de forma separada. Os resultados da figura 18 mostram as linhas detetadas sob um fundo branco, que representa a região de interesse.

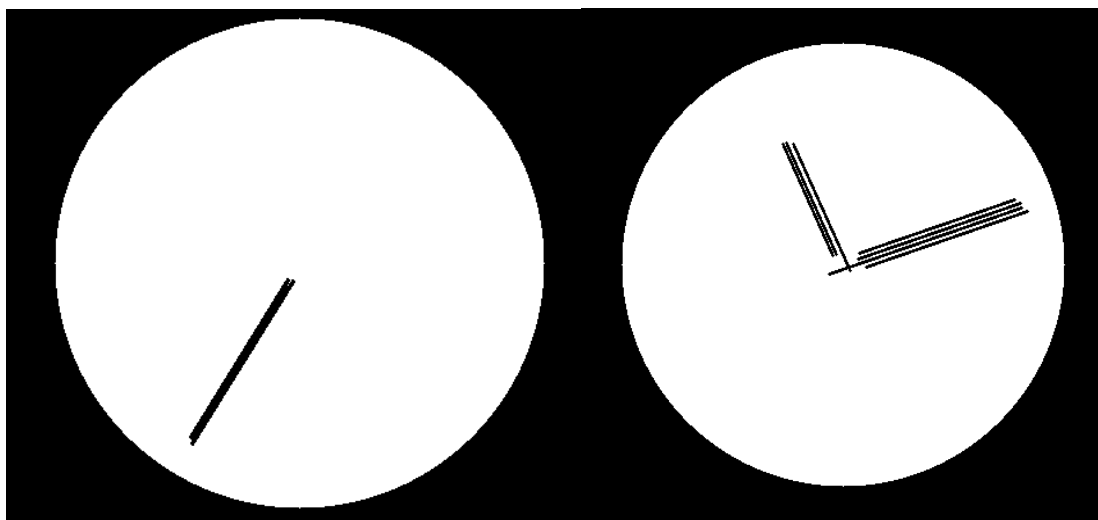


FIGURA 18 – LINHAS DETETADAS NAS IMAGENS DA ESCALA S (À DIREITA) E DA ESCALA V (À ESQUERDA)

A função “detetar_ponteiros” para as imagens utilizadas a partir da escala de cores HSV acaba por ser diferente para as duas escalas utilizadas, dando destaque para a variável “ponteiro_segundos” que é uma *flag* para a execução da parte em que são guardadas as linhas no array “hands”. Caso esta *flag* esteja ativada (isto é, quando a função é aplicada para a imagem em escala S), não irá ser considerado se a espessura da linha é positiva ou negativa, e irá diretamente guardar os dados dentro do array.


```

if ponteiro_segundos is False:
    # If the thickness is greater than 0, it means there are at least two parallel lines
    if max_thickness > 0:
        # Add this set to the clock hands list
        hands.append(line)
    else:
        hands.append(line)

```

FIGURA 19 – APLICAÇÃO DA FLAG “PONTEIRO_SEGUNDOS”

Para identificar os ponteiros, são utilizadas as funções “get_hands_S” (para o ponteiro dos segundos) e “get_hands_V” (para os ponteiros dos minutos e das horas). A diferença entre estas funções e a função “get_hands” utilizada para a escala de cinzas, é o facto de as duas funções apenas organizarem os seus ponteiros por comprimento, uma vez que o ponteiro dos segundos se encontra isolado numa imagem à parte.

Por fim, o processo de obtenção dos ângulos de cada ponteiro, o respetivo tempo e o método para desenhar os retângulos a identificar os ponteiros e o tempo na imagem original continuam a ser os mesmos que a escala de cinzas.

4. Execução do algoritmo em vídeo

Para o algoritmo funcionasse bem com um vídeo, foram necessárias algumas modificações, de modo a ultrapassar dificuldades que apareceram durante as primeiras execuções do vídeo. De início, foi escolhido a escala de cinzas para a execução do vídeo. No entanto, o facto de os três ponteiros estarem a ser detetados numa imagem, revela ser um obstáculo, uma vez que caso os três ponteiros ou apenas dois deles estiverem sobrepostos, não haveria uma maneira que fosse mais fácil de distinguir os ponteiros. A vantagem de o ponteiro dos segundos estar isolado dos restantes na extração do tempo analógico em escala de cores HSV, traz essa facilidade de ultrapassar os problemas relacionados com a sobreposição de ponteiros, uma vez que é sempre garantido ter o tempo dos segundos correto. Isto pode-se provar na função “get_hands_V”, onde caso o número de ponteiros detetados for “1” (if len(sorted_hands_by_length) == 1), vai atribuir o único valor que existe aos ponteiros dos minutos e das horas. Caso não seja detetado nenhum ponteiro (if hands == []), irá ser atribuído o valor do ponteiro dos segundos aos restantes ponteiros, tal como indica a figura 20.

```
# Arrange the remaining 2 clock hands by length
sorted_hands_by_length = sorted(hands, key=lambda hands: hands[2])

if len(sorted_hands_by_length) == 1:
    test2 = True
    minute_hand = sorted_hands_by_length[0]
    # The hour hand is the hand with the shortest length and the remaining hand is the minute hand
    hour_hand = sorted_hands_by_length[0]
else:
    if hands != []:
        test2 = False
        hour_hand = sorted_hands_by_length[0]
        minute_hand = sorted_hands_by_length[1]
    else:
        if second_hand is not None:
            hour_hand = second_hand
            minute_hand = second_hand

return hour_hand, minute_hand
```

FIGURA 20 – FUNÇÃO “GET_HANDS_V”

Pelo facto de algumas *frames* do vídeo não detetarem bem os ponteiros por causa de uma sobreposição não total (e é de notar que acontecia frequentemente na imagem de escala V, em que a parte de trás do ponteiro dos segundos, por ter uma espessura grande, sobrepunha com o ponteiro dos minutos ou dos segundos), havia problemas na função “agrupar_linhas”, que não conseguia verificar a condição principal da relação entre o máximo ou o mínimo da distância entre os pontos da linha mais longe do centro do círculo e o próprio centro do círculo. Para ultrapassar esta dificuldade, foi criada uma alternativa para as linhas que não cumpriam as condições desejadas (ver figura 20), ao calcular o centroide das linhas e adicionar os dados das linhas e do centroide de cada uma delas no *array* “grupos”, com o valor de ângulo (“angle”) a “0”.

```
else:
    centroid = ((x1 + x2) / 2, (y1 + y2) / 2)
    # Handle special case where condition is not met
    grupos.append({'lines': [line], 'angle': 0, 'centroid': centroid})
```

FIGURA 21 – ALTERNATIVA UTILIZADA NA FUNÇÃO “AGRUPAR_LINHAS”

Para além destas modificações, foi necessário aumentar os valores dos parâmetros da função “Canny” relacionados com os limiares máximos de *threshold* da própria função nas escalas S e V. A figura 22, mostra em comentário os valores originais antes da implementação.

```
edges_S = cv2.Canny(blurred_S, 50, 180) #150
edges_V = cv2.Canny(blurred_V, 50, 195) #150
```

FIGURA 22 – CHAMADA DA FUNÇÃO “CANNY” COM OS LIMIARES MÁXIMOS ALTERADOS

Estas modificações, a aplicação do algoritmo no vídeo ficou funcional, porém com alguns erros que podem ser visualizados no ficheiro .csv, demonstrado na figura 23.

#Group 6,2230075,João Paulino,2232626,Judá Imbó				
Frame 1,00:59:00				
Frame 2,00:01:01				
Frame 3,00:01:02				
Frame 4,00:01:03				
Frame 5,00:01:04				
Frame 6,00:02:05				
Frame 7,00:02:07				
Frame 8,00:02:08				
Frame 9,00:03:09				
Frame 10,00:03:10				
Frame 11,00:04:12				
Frame 12,00:03:13				

FIGURA 23 – FICHEIRO .CSV COM AS FRAMES E O TEMPO OBTIDO

5. Conclusão

Este trabalho contribuiu para a compreensão das matérias relacionadas com o processamento de imagem e a extração de informação a partir de diferentes escalas de cor. Houve algumas dificuldades na execução do algoritmo no vídeo, como não se tinha testado imagens com diferentes posições dos ponteiros. Pode-se aprender com as sobreposições e o facto de a obstrução total ou parcial do objeto de interesse pode afetar o algoritmo e o seu resultado final. Isso reflete-se também no facto de terem aparecido diversos problemas em diferentes partes do algoritmo, apesar destas terem sido ultrapassadas. Apesar de o programa se encontrar funcional, sabe-se que poderiam ser implementadas algumas melhorias, principalmente, na simplificação geral do algoritmo.